

Σύστημα Διάχυσης Μηνυμάτων με Διαμοιραζόμενη Μνήμη

Αναλυτική Αναφορά Υλοποίησης

Μάθημα: Λειτουργικά Συστήματα (Κ22)

Εξάμηνο: 2025–2026

Τίτλος Εργασίας: Πρώτη Προγραμματιστική Εργασία

Όνομα: ADAM AHMED

A.M: 1115202400225

1. Εισαγωγή

1.1 Σκοπός της Εργασίας

Η εργασία αυτή αφορά την ανάπτυξη ενός συστήματος διάχυσης μηνυμάτων μεταξύ διεργασιών σε περιβάλλον Unix, με αποκλειστική χρήση:

- System V διαμοιραζόμενης μνήμης (`shmget`, `shmat`, `shmdt`, `shmctl`)
- System V σηματοφόρων (`semget`, `semop`, `semctl`)

Δεν χρησιμοποιούνται άλλες μορφές IPC, όπως:

- POSIX shared memory ή semaphores
- sockets, pipes ή αρχεία

Η επιλογή της System V IPC διασφαλίζει ότι όλες οι διεργασίες επικοινωνούν μέσω κοινής περιοχής μνήμης, με κατάλληλο συγχρονισμό.

1.2 Βασικές Προδιαγραφές

1.2.1 Διάλογοι

Οι διεργασίες επικοινωνούν μέσω «διαλόγων» — ανεξάρτητων καναλιών ανταλλαγής μηνυμάτων με τα εξής χαρακτηριστικά:

- Κάθε διάλογος έχει μοναδικό αναγνωριστικό (ID), που καθορίζεται από τη διεργασία που τον δημιουργεί.
- Τουλάχιστον δύο διεργασίες πρέπει να συμμετέχουν σε κάθε διάλογο.
- Το σύστημα καταγράφει τον αριθμό και τα αναγνωριστικά των συμμετεχόντων.

- Κάθε διεργασία μπορεί να **αποστέλλει και να λαμβάνει** μηνύματα ταυτόχρονα.

1.2.2 Μηνύματα

Τα μηνύματα που ανταλλάσσονται έχουν τα εξής χαρακτηριστικά:

- Αποθηκεύονται σε **μία κοινή διαμοιραζόμενη μνήμη** (κοινή για όλους τους διαλόγους).
- Περιλαμβάνουν το **ID του διαλόγου** στον οποίο ανήκουν.
- Το περιεχόμενο τους είναι **απλό κείμενο**, έως 256 χαρακτήρες.
- Παραδίδονται **ακριβώς μία φορά** σε κάθε συμμετέχουσα διεργασία.
- **Διαγράφονται** από τη μνήμη μόνο όταν όλες οι διεργασίες έχουν ολοκληρώσει την ανάγνωση.

1.2.3 Τερματισμός Διαλόγου

Η ολοκλήρωση ενός διαλόγου πραγματοποιείται όταν μια διεργασία στείλει μήνυμα με περιεχόμενο **TERMINATE**. Σε αυτή την περίπτωση:

- Όλες οι διεργασίες που συμμετέχουν στον διάλογο διακόπτουν την αναμονή για μηνύματα.
- Η διεργασία που έστειλε το **TERMINATE** επίσης τερματίζει.
- Όταν ολοκληρωθεί και ο τελευταίος διάλογος:
 - Απελευθερώνονται οι σημαφόροι (**semctl(..., IPC_RMID)**).
 - Αποδεσμεύεται η διαμοιραζόμενη μνήμη (**shmctl(..., IPC_RMID)**).

1.3 Δομή της Παρούσας Αναφοράς

Στην παρούσα τεκμηρίωση παρουσιάζονται:

1. Ο σχεδιασμός των βασικών δομών δεδομένων και οι σχετικές αποφάσεις.
2. Ο τρόπος συγχρονισμού μεταξύ διεργασιών με χρήση σημαφόρων.
3. Το πρωτόκολλο λειτουργίας για τις βασικές εντολές (**create, join, send, recv, TERMINATE**).
4. Η πορεία της υλοποίησης και τα προβλήματα που αντιμετωπίστηκαν.
5. Παραδείγματα χρήσης και σενάρια δοκιμών.
6. Συμπεράσματα και πιθανές μελλοντικές βελτιώσεις.

2. Σχεδιασμός Δομών Δεδομένων

2.1 Αρχιτεκτονική Συστήματος

Η υλοποίηση βασίζεται σε μία **κοινή δομή διαμοιραζόμενης μνήμης**, η οποία περιλαμβάνει:

- Έναν **κυκλικό buffer** για την αποθήκευση έως 100 μηνυμάτων.
- Έναν **πίνακα διαλόγων** χωρητικότητας έως 10 ενεργών διαλόγων.

- Διάφορα **μεταδεδομένα**, όπως δείκτες θέσης, μετρητές, και αναγνωριστικά σηματοφόρων.

Πλεονεκτήματα σχεδιασμού:

- **Απλότητα**: Όλες οι διεργασίες προσπελαίνουν την ίδια μνήμη με ένα μόνο `shmat`.
- **Συνεκτικότητα**: Όλες οι δομές βρίσκονται συγκεντρωμένες σε ένα σημείο, διευκολύνοντας τη διαχείριση και την αποσφαλμάτωση.
- **Προβλεψιμότητα**: Το μέγεθος της μνήμης είναι σταθερό και γνωστό εκ των προτέρων.
- **Ασφάλεια**: Ολόκληρη η δομή προστατεύεται με έναν mutex σηματοφόρο.

Το συνολικό μέγεθος της μνήμης εκτιμάται περίπου στις 30.584 bytes.

2.2 Δομή Μηνύματος (**Message**)

Κάθε μήνυμα έχει συγκεκριμένα πεδία ώστε να διασφαλίζεται η μοναδική ανάγνωσή του από κάθε συμμετέχουσα διεργασία:

```
typedef struct {
    int dialogue_id;
    pid_t sender_pid;
    char content[256];
    int sequence_number;
    time_t timestamp;
    int is_active;
    unsigned int read_bitmap;
    int total_readers_expected;
} Message;
```

Πεδία:

- **dialogue_id**: Αναγνωρίζει σε ποιον διάλογο ανήκει το μήνυμα.
- **sender_pid**: Το PID του αποστολέα — χρήσιμο για έλεγχο και αποσφαλμάτωση.
- **content**: Το περιεχόμενο του μηνύματος (απλό κείμενο).
- **sequence_number**: Αύξουσα αρίθμηση εντός κάθε διαλόγου για έλεγχο σειράς.
- **timestamp**: Η χρονική στιγμή αποστολής.
- **is_active**: Δείκτης κατάστασης (0: κενό, 1: ενεργό μήνυμα).
- **read_bitmap**: Bitmap 32-bit που δείχνει ποιοι συμμετέχοντες έχουν διαβάσει το μήνυμα.
- **total_readers_expected**: Ο αριθμός συμμετεχόντων τη στιγμή της αποστολής — χρησιμοποιείται για να προσδιοριστεί πότε ένα μήνυμα μπορεί να αφαιρεθεί.

Η χρήση του `read_bitmap` επιτρέπει αποδοτικό έλεγχο ανάγνωσης ανά διεργασία, εξασφαλίζοντας μοναδική παράδοση χωρίς επανάληψη.

2.3 Δομή Διαλόγου (Dialogue)

Κάθε διάλογος ορίζεται ως εξής:

```
typedef struct {
    int dialogue_id;
    int num_participants;
    pid_t participant_pids[20];
    int is_active;
    int next_sequence_number;
} Dialogue;
```

Πεδία:

- `dialogue_id`: Ο μοναδικός αριθμός του διαλόγου.
- `num_participants`: Το πλήθος των ενεργών συμμετεχόντων.
- `participant_pids[ ]`: Πίνακας με τα PIDs των συμμετεχόντων.
- `is_active`: Κατάσταση του διαλόγου (1: ενεργός, 0: τερματισμένος).
- `next_sequence_number`: Επόμενος αριθμός σειράς για τα εισερχόμενα μηνύματα.

Η θέση (index) κάθε PID στον πίνακα αντιστοιχεί σε bit στο `read_bitmap` των μηνυμάτων, εξασφαλίζοντας συσχέτιση ανάγνωσης.

2.4 Δομή Διαμοιραζόμενης Μνήμης (SharedMemory)

Η κεντρική δομή που αποθηκεύεται στη System V διαμοιραζόμενη μνήμη:

```
typedef struct {
    Message messages[100];
    Dialogue dialogues[10];
    int message_write_index;
    int num_active_dialogues;
    int active_reader_count;

    int mutex_semid;
    int msg_available_semid;
    int slot_available_semid;
```

```
} SharedMemory;
```

Πεδία:

- `messages[ ]`: Κυκλικός buffer για τα μηνύματα.
- `dialogues[ ]`: Πίνακας ενεργών διαλόγων.
- `message_write_index`: Δείκτης κυκλικής εγγραφής στο buffer.
- `num_active_dialogues`: Πλήθος ενεργών διαλόγων (για παρακολούθηση κατάστασης).
- `active_reader_count`: Χρήσιμο για τον καθορισμό πότε μπορεί να γίνει πλήρης καθαρισμός (τελευταίος αναγνώστης).

Σημαφόροι:

- `mutex_semid`: Χρησιμοποιείται ως mutex για την προστασία κρίσιμων περιοχών.
- `msg_available_semid`: Μετρητικός σημαφόρος που δηλώνει πόσα νέα μηνύματα υπάρχουν.
- `slot_available_semid`: Δηλώνει τον αριθμό των διαθέσιμων θέσεων στον buffer.

Τα IDs των σημαφόρων αποθηκεύονται εντός της μνήμης για εύκολη πρόσβαση από όλες τις διεργασίες.

3. Συγχρονισμός και Παράδοση Μηνυμάτων «Ακριβώς Μία Φορά»

3.1 Γενική Αρχιτεκτονική

Ο συγχρονισμός των διεργασιών επιτυγχάνεται μέσω **τριών σημαφόρων System V**, οι οποίοι διασφαλίζουν την ορθή και αποδοτική συνεργασία μεταξύ αποστολέων και αναγνωστών:

- `mutex_semid`: Δυναμικός σημαφόρος για την προστασία κρίσιμων περιοχών (mutex).
- `msg_available_semid`: Μετρητικός σημαφόρος για την ένδειξη ύπαρξης νέων μηνυμάτων.
- `slot_available_semid`: Μετρητικός σημαφόρος για την παρακολούθηση ελεύθερων θέσεων στον buffer.

Αυτοί οι μηχανισμοί συγχρονισμού διασφαλίζουν:

- Ασφαλή πρόσβαση σε κοινές δομές (π.χ. πίνακες, bitmaps).
 - Αποδοτικό χειρισμό των μηνυμάτων χωρίς απώλειες ή διπλοεγγραφές.
 - Αποφυγή υπερχείλισης του buffer.
-

3.2 Περιγραφή Σημαφόρων

3.2.1 `mutex_semId`

- **Ρόλος:** Παρέχει αποκλειστική πρόσβαση σε κρίσιμες περιοχές.
- **Αρχική τιμή:** 1 (διαθέσιμο).
- **Χρήση:** Περιβάλλει λειτουργίες εγγραφής/ανάγνωσης μηνυμάτων, επεξεργασίας διαλόγων και ενημέρωσης shared μεταβλητών.

Η χρήση του είναι απαραίτητη για την αποφυγή race conditions μεταξύ διεργασιών.

3.2.2 `msg_available_semId`

- **Ρόλος:** Δηλώνει ότι υπάρχουν νέα μηνύματα προς ανάγνωση.
- **Αρχική τιμή:** 0.
- **Χρήση:** Αυξάνεται από αποστολείς κατά την εγγραφή νέου μηνύματος. Οι αναγνώστες πραγματοποιούν μη μπλοκάροντα `semop`, ελέγχουν για νέα μηνύματα και, αν δεν υπάρχουν, περιμένουν με `usleep()` για να αποφύγουν άσκοπη χρήση CPU.

Συνδυασμός μη μπλοκάροντος ελέγχου και ύπνου εξισορροπεί την απόδοση και την απόκριση.

3.2.3 `slot_available_semId`

- **Ρόλος:** Διασφαλίζει ότι δεν ξεπερνιέται η χωρητικότητα του buffer.
- **Αρχική τιμή:** 100 (το μέγιστο πλήθος μηνυμάτων).
- **Χρήση:** Πριν την αποστολή μηνύματος, κάθε διεργασία κάνει `sem_down`. Όταν ένα μήνυμα έχει αναγνωστεί από όλους, γίνεται `sem_up`.

Αυτός ο σημαφόρος εφαρμόζει "πίεση προς τα πίσω" (backpressure), αποτρέποντας την ανεξέλεγκτη συσσώρευση μηνυμάτων.

3.3 Παράδοση «Ακριβώς Μία Φορά»

Η αποστολή και η λήψη κάθε μηνύματος ακολουθούν συγκεκριμένο πρωτόκολλο για να διασφαλιστεί η μοναδική ανάγνωση από κάθε συμμετέχουσα διεργασία.

3.3.1 Λειτουργία Μηχανισμού

1. **Κατά την αποστολή:**
 - Το `read_bitmap` μηδενίζεται.

- Το `total_readers_expected` λαμβάνει την τρέχουσα τιμή των συμμετεχόντων στον διάλογο.
- 2. **Κατά την ανάγνωση:**
 - Κάθε διεργασία εντοπίζει το index της στον πίνακα `participant_pids`.
 - Ελέγχει το αντίστοιχο bit στο `read_bitmap`.
 - Αν είναι ήδη 1, το μήνυμα αγνοείται.
 - Αν είναι 0, το bit ενεργοποιείται και το μήνυμα διαβάζεται.
- 3. **Κατά την απελευθέρωση:**
 - Αν το πλήθος ενεργών bits ισούται με `total_readers_expected`, τότε:
 - Το `is_active` γίνεται 0.
 - Το περιεχόμενο εκκαθαρίζεται.
 - Αυξάνεται ο `slot_available_semid`.

Η χρήση mutex διασφαλίζει ότι αυτές οι τροποποιήσεις γίνονται με ασφάλεια, χωρίς επικαλύψεις.

3.3.2 Παράδειγμα Χρήσης

Έστω διάλογος με τρεις συμμετέχοντες (PID 1001, 1002, 1003). Η αποστολή μηνύματος θέτει το `read_bitmap` σε `0b000`.

- Ο πρώτος αναγνώστης (index 0) θέτει το bit 0 → `0b001`.
- Ο δεύτερος (index 1) θέτει το bit 1 → `0b011`.
- Ο τρίτος (index 2) θέτει το bit 2 → `0b111`.

Μόλις το `read_bitmap` γίνει ίσο με τον αριθμό συμμετεχόντων, το μήνυμα απελευθερώνεται.

Μελλοντικές επανεισερχόμενες διεργασίες δεν επηρεάζουν το μήνυμα, αφού `total_readers_expected` παραμένει σταθερό κατά την αποστολή.

3.3.3 Εγγυήσεις Ορθότητας

- **Μοναδική Ανάγνωση:** Το `read_bitmap` εγγυάται ότι κάθε PID διαβάζει μία φορά.
- **Μη Απώλεια:** Κανένα μήνυμα δεν αφαιρείται πριν διαβαστεί από όλους.
- **Ασφαλής Συγκατοίκηση:** Όλες οι τροποποιήσεις bitmap γίνονται υπό mutex.
- **Ελεγχόμενη Χρήση Buffer:** Ο μετρητικός σηματοδότης θέσεων προφυλάσσει από υπερχείλιση.

4. Πρωτόκολλο Λειτουργίας

4.1 Αρχικοποίηση Συστήματος (`./shm init`)

Η εντολή `init` εκτελείται μία φορά για τη δημιουργία των βασικών πόρων:

- Δημιουργία διαμοιραζόμενης μνήμης.
 - Αρχικοποίηση πινάκων μηνυμάτων και διαλόγων.
 - Ρύθμιση των τριών σημαφόρων:
 - `mutex`: για αποκλειστική πρόσβαση σε κρίσιμες περιοχές.
 - `msg_available`: για ειδοποίηση νέων μηνυμάτων.
 - `slot_available`: για παρακολούθηση ελεύθερων θέσεων στον buffer.
-

4.2 Δημιουργία Διαλόγου (`./shm create <dialogue_id>`)

Η δημιουργία νέου διαλόγου γίνεται εντός `mutex`:

- Αναζητείται διαθέσιμη θέση στον πίνακα διαλόγων.
 - Αρχικοποιείται με `dialogue_id`, το PID του δημιουργού και μηδενικό αριθμό μηνυμάτων.
 - Αν υπάρχει ήδη, η διεργασία αποτυγχάνει.
-

4.3 Συμμετοχή σε Διάλογο (`./shm join <dialogue_id>`)

Η διεργασία εντάσσεται σε υπάρχοντα διάλογο, εφόσον:

- Δεν είναι ήδη συμμετέχουσα.
- Υπάρχει διαθέσιμη θέση (μέχρι `MAX_PARTICIPANTS`).

Οι νέοι συμμετέχοντες **δεν λαμβάνουν** παλαιότερα μηνύματα, καθώς κάθε μήνυμα διατηρεί τον αριθμό αναγνωστών τη στιγμή αποστολής του.

4.4 Αποστολή Μηνύματος (`./shm send <dialogue_id> "<message>"`)

Η διαδικασία αποστολής περιλαμβάνει:

1. Αναμονή για διαθέσιμη θέση στον buffer.
2. Κρίσιμη περιοχή όπου:
 - Εντοπίζεται ο διάλογος.
 - Γράφεται το νέο μήνυμα σε θέση του buffer.
 - Ενημερώνεται ο δείκτης εγγραφής.
3. Ενεργοποίηση του σημαφόρου νέου μηνύματος (`msg_available`).

Αν σταλεί **"TERMINATE"**, ενεργοποιείται ειδική διαδικασία για τον τερματισμό του διαλόγου και την αποδέσμευση των πόρων.

4.5 Λειτουργία Αναγνώστη (**./shm recv**)

Κάθε διεργασία σε λειτουργία αναγνώστη:

- Εγγράφεται αυτόματα σε ενεργούς διαλόγους (αν δεν συμμετέχει ήδη).
- Εισέρχεται σε βρόχο αναμονής και επεξεργασίας μηνυμάτων.
- Ελέγχει αν έχει ήδη διαβάσει κάθε μήνυμα μέσω **read_bitmap**.
- Αντιγράφει, εμφανίζει και σημειώνει το μήνυμα ως διαβασμένο.
- Όταν όλα τα bits του μηνύματος έχουν οριστεί, το slot ελευθερώνεται.

Αν ληφθεί μήνυμα **"TERMINATE"**, η διεργασία εξέρχεται του βρόχου και ελέγχει αν πρέπει να καθαρίσει τους πόρους (αν είναι ο τελευταίος ενεργός αναγνώστης).

4.6 Τερματισμός Διαλόγου (**TERMINATE**)

Ο τερματισμός γίνεται συντονισμένα:

- Ο **αποστολέας** στέλνει το μήνυμα **TERMINATE** και αποχωρεί.
- Οι **αναγνώστες** τερματίζουν αφού λάβουν το μήνυμα και εκτελούν **leave_dialogue**.
- Ο **τελευταίος αναγνώστης** καθαρίζει τη μνήμη και τους σηματοφόρους.

Αυτό εξασφαλίζει ομαλό και πλήρη καθαρισμό χωρίς διαρροές πόρων.

5. Πορεία Υλοποίησης & Δυσκολίες

5.1 Σχεδιασμός Δομών

Την πρώτη εβδομάδα έγινε ανάλυση απαιτήσεων και σχεδιάστηκαν οι βασικές δομές: **Message**, **Dialogue**, **SharedMemory**. Ορίστηκαν πρακτικά όρια (100 μηνύματα, 10 διάλογοι, 20 συμμετέχοντες ανά διάλογο, 256 χαρακτήρες ανά μήνυμα) ώστε να είναι επαρκή για δοκιμές χωρίς υπερβολική κατανάλωση μνήμης (~30KB συνολικά).

Επιλέχθηκε κυκλικός buffer και χρήση bitmap για exact-once delivery, αντί απλών counters, ώστε να διασφαλιστεί ακρίβεια ανά διεργασία.

5.2 Υλοποίηση Υποσυστήματος IPC

Στη δεύτερη εβδομάδα υλοποιήθηκε η δημιουργία και προσπέλαση της διαμοιραζόμενης μνήμης και των σημαφόρων. Έγιναν περιτυλίξεις γύρω από τις κλήσεις `semctl` και `semop` για απλοποίηση της χρήσης. Δοκιμάστηκε η πρόσβαση από πολλαπλές διεργασίες και επιβεβαιώθηκε η σωστή λειτουργία μέσω ενός απλού σχήματος παραγωγού-καταναλωτή.

Ένα μικρό εμπόδιο παρουσιάστηκε στο macOS με τον ορισμό του `union semun`, που λύθηκε με conditional compilation.

5.3 Διαχείριση Διαλόγων

Την τρίτη εβδομάδα υλοποιήθηκαν οι συναρτήσεις `create`, `join` και `leave` για τους διαλόγους. Ιδιαίτερη προσοχή δόθηκε στην προστασία από ταυτόχρονες δημιουργίες ίδιου `dialogue_id` μέσω `mutex`. Αντιμετωπίστηκε και το πρόβλημα με "νεκρά" PIDs, εφαρμόζοντας έλεγχο με `kill(pid, 0)` πριν τη διατήρησή τους.

5.4 Κυκλικός Buffer & Αποστολή Μηνυμάτων

Η τέταρτη εβδομάδα επικεντρώθηκε στη διαχείριση του buffer μηνυμάτων. Υλοποιήθηκε αποδοτική αναζήτηση θέσης με κυκλικό τρόπο και ελέγχθηκε η σωστή σειρά λήψης σημαφόρων, ώστε να αποφευχθούν πιθανοί deadlocks. Το σύστημα `slot_available` εξασφαλίζει ότι δεν υπερχειλίζει ο buffer.

5.5 Exact-Once Παράδοση

Στην πέμπτη εβδομάδα ολοκληρώθηκε ο μηχανισμός παράδοσης κάθε μηνύματος ακριβώς μία φορά σε κάθε συμμετέχοντα. Υλοποιήθηκε `read_bitmap` και υπολογισμός του πλήθους αναγνωστών με `popcount`. Όλες οι τροποποιήσεις γίνονται εντός `mutex` για αποφυγή race conditions. Έγιναν εκτενείς δοκιμές με πολλαπλούς αναγνώστες και ελεγχόμενες καθυστερήσεις.

5.6 Τερματισμός & Καθαρισμός

Η έκτη εβδομάδα αφιερώθηκε στη διαχείριση `TERMINATE`. Όταν σταλεί μήνυμα `TERMINATE`, οι αναγνώστες τερματίζουν και ο τελευταίος αποδεσμεύει όλους τους πόρους (μνήμη και σημαφόρους). Εισήχθη μετρητής `active_reader_count` για να εντοπίζεται πότε ο τελευταίος αναγνώστης αποχωρεί. Αν ο αποστολέας τερματίσει πρώτος, δίνει χρόνο στους αναγνώστες να καθαρίσουν.

5.7 Testing & Debugging

Την τελευταία εβδομάδα πραγματοποιήθηκαν εντατικές δοκιμές με σενάρια ταυτόχρονης αποστολής, πολλών διαλόγων, γεμάτου buffer, και αναγνωστών που εισέρχονται/εξέρχονται ασύγχρονα. Χρησιμοποιήθηκαν βοηθητικά scripts και `printf` για εντοπισμό λαθών.

Το σύστημα απέδωσε ικανοποιητικά, χωρίς διαρροές ή ορφανούς πόρους, και αποδείχθηκε σταθερό για την προβλεπόμενη χρήση του.

6. Παραδείγματα Χρήσης & Δοκιμές

6.1 Βασικό Σενάριο: 2 Αναγνώστες, 1 Αποστολέας

Το σενάριο αυτό επιδεικνύει την πλήρη ροή λειτουργίας του συστήματος:

- Ο δημιουργός ξεκινά το σύστημα και δημιουργεί έναν διάλογο.
- Ένας δεύτερος αναγνώστης συμμετέχει.
- Ένας αποστολέας στέλνει δύο μηνύματα και στη συνέχεια ένα μήνυμα `TERMINATE`.

Αποτελέσματα:

- Και οι δύο αναγνώστες λαμβάνουν όλα τα μηνύματα, με την ίδια σειρά και ακριβώς μία φορά.
 - Το `TERMINATE` οδηγεί σε τερματισμό και καθαρισμό όλων των πόρων.
 - Επαληθεύεται μέσω `ipcs` ότι δεν υπάρχουν πλέον πόροι (shared memory ή semaphores).
-

6.2 Edge Case: Πρόληψη Υπερχείλισης Buffer

Σε αυτό το σενάριο, 101 αποστολές προσπαθούν να στείλουν ταυτόχρονα:

- Οι 100 πρώτοι αποκτούν πρόσβαση, καταλαμβάνοντας όλες τις διαθέσιμες θέσεις.
- Ο 101ος αποστολέας μπλοκάρει (αναμένει διαθέσιμο slot).

Επαλήθευση:

- Δεν χάνεται κανένα μήνυμα.
 - Το σύστημα εφαρμόζει «backpressure» και προστατεύεται από υπερχείλιση.
-

6.3 Edge Case: Καθυστερημένος Συμμετέχων

Σε αυτό το σενάριο:

1. Ένας διάλογος δημιουργείται με 2 συμμετέχοντες.

2. Αποστέλλονται 5 μηνύματα.
3. Ενώνεται 3ος συμμετέχων.
4. Αποστέλλεται νέο μήνυμα.

Αποτελέσματα:

- Ο τρίτος συμμετέχων δεν λαμβάνει τα 5 πρώτα μηνύματα.
- Λαμβάνει μόνο τα επόμενα μηνύματα που αποστέλλονται μετά την είσοδό του.

Ορθότητα: Το σύστημα λειτουργεί όπως ορίζεται—η λίστα αναμενόμενων αναγνωστών καθορίζεται κατά την αποστολή, όχι κατά την ανάγνωση.

6.4 Edge Case: Πολλαπλοί Διάλογοι

Σε αυτό το σενάριο:

- Υπάρχουν δύο διάλογοι με διαφορετικούς συμμετέχοντες.
- Μία διεργασία συμμετέχει και στους δύο.

Αποτελέσματα:

- Η διεργασία λαμβάνει μηνύματα και από τους δύο διαλόγους, χωρίς σύγχυση.
- Τα μηνύματα διαχωρίζονται με βάση το `dialogue_id`.

7. Συμπεράσματα & Μαθήματα

7.1 Τι Έμαθα

- **System V IPC:** Απόκτηση πρακτικής εμπειρίας με shared memory και semaphores, κατανόηση των βασικών διαφορών με το POSIX API και σωστή διαχείριση των IPC πόρων.
- **Συγχρονισμός:** Σχεδίαση σωστής χρήσης σημαφόρων για αποφυγή race conditions, διαχείριση deadlocks και επίτευξη ακριβούς παραλαβής μηνυμάτων (exact-once).
- **Σχεδιασμός Συστημάτων:** Ισορροπία μεταξύ απόδοσης, σωστής λειτουργίας και απλότητας μέσα από τις επιλογές δομών και πρωτοκόλλων.

7.2 Κύριες Προκλήσεις

- **Exact-Once Delivery:** Η μετάβαση από απλό counter σε bitmap αποτέλεσε σημαντική αλλαγή που έλυσε το πρόβλημα ορθότητας.
- **Καθαρισμός Πόρων:** Συντονισμός μεταξύ αποστολέα και αναγνωστών ώστε να μην καταστρέφονται πόροι πρόωρα.
- **Διαχείριση Buffer:** Προσθήκη μηχανισμών πίεσης (backpressure) ώστε να αποφεύγεται η υπερχείλιση.

7.3 Πιθανές Βελτιώσεις

- Υιοθέτηση **POSIX IPC** για ευρύτερη συμβατότητα.
- Προσθήκη **επιμονής (persistence)** των μηνυμάτων με αποθήκευση σε αρχείο.
- Εισαγωγή **μηχανισμών ταυτοποίησης (authentication)**.
- Υποστήριξη **δικτύωσης**, ώστε να μπορούν οι χρήστες να επικοινωνούν από διαφορετικά μηχανήματα.
- Καλύτερη **διαχείριση ασφαμάτων** και επεξηγηματικά μηνύματα.

7.4 Τελικό Αποτέλεσμα

Το σύστημα:

-  Εξασφαλίζει συγχρονισμό και exact-once delivery.
-  Διαχειρίζεται σωστά τους διαθέσιμους πόρους.
-  Καλύπτει πολλαπλά σενάρια και edge cases.
-  Είναι επεκτάσιμο και κατανοητό.

Η εργασία αυτή αποτέλεσε σημαντική ευκαιρία πρακτικής εξάσκησης στον συγχρονισμό διεργασιών και στην αρχιτεκτονική συστημάτων IPC.